

A REPORT
ON
QuickChat
Offered by the
Department Computer Science and Engineering
School of Engineering and Sciences

Submitted by

T Yuva Sai Asrith	AP23110010708
K.Anishitha	AP23110011039
T Kiraneswar	AP23110010093
B Kesava Rama Sanjeev	AP23110010604



SRM University-AP
Neerukonda, Mangalagiri, Guntur
Andhra Pradesh – 522 240
December, 2025

QuickChat(React)

Introduction:

Step into the future of instant communication with **QuickChat** – a revolutionary chat experience built using the power of **React.js** and **Firebase**. Blending modern technology with a user-first design, QuickChat is here to change the way you connect, collaborate, and chat in real time.

Created for today's fast communicator, QuickChat brings together strong real-time messaging features with a clean, distraction-free interface. Whether it's a quick study chat or a remote team discussion, the platform helps you stay focused and engaged, giving you a smooth and enjoyable conversation experience.

At the heart of QuickChat lies **React**, a fast and flexible JavaScript library, paired with Firebase's **Authentication** and **Firestore** for secure sign-in and instant message updates. Enjoy a beautiful, responsive design where messages, avatars, and room interactions feel quick and natural. No matter what device you use—desktop, tablet, or smartphone—the experience stays consistent and visually appealing.

Say goodbye to the limitations of old messaging platforms. With QuickChat, you enter a world full of new possibilities in real-time communication. Join us on this exciting journey as we reshape how people interact online. Get ready to upgrade your chat experience—just click **“Join Room”** and step into a new era of instant connection.

Scenario-Based Intro:-

Imagine it's the day before your group project submission. Everyone on your team is busy in different places some are in the library, some in their hostel rooms, and others commuting yet all of you need one simple space to coordinate without messy group chats or long video calls.

You open QuickChat in your browser. With a single click, you sign in using your Google account, no registration forms, no password creation. QuickChat greets you with a clean, modern interface built with Tailwind CSS and DaisyUI, themed to match your preference.

You click "Create New Room," and instantly, QuickChat generates a short, human-readable room ID: sunny-hello-42. You copy it and paste it into your team's group message. Within seconds, your teammates join the room from their laptops and phones.

As the conversation begins, messages appear in real time, powered by Firebase Firestore. A question is asked, a design mockup link is shared, a code snippet is pasted and everyone sees it instantly. Each message shows the sender's name and avatar, making it easy to follow who's contributing, even in a fast-moving discussion.

Later, you decide to split into smaller groups. You create a second room for the backend team and share a new room ID. The same seamless experience repeats: quick joining, instant messaging, and clear visual identity for each participant.

QuickChat handles the technical complexity authentication, real-time syncing, responsive layout so you and your team can focus entirely on the conversation. No setup hassle, no server maintenance, just pure, focused communication in dedicated, shareable rooms.

Target Audience:-

QuickChat is tailored for a diverse community of individuals, including:

Students and Study Groups: Learners needing real-time spaces to discuss assignments and collaborate on projects.

Remote Teams and Professionals: Small teams requiring lightweight channels for coordination and brainstorming.

Communities and Social Groups: Friends and clubs seeking simple, private chat rooms for conversations and planning.

Project Goals and Objectives:-

The primary goal of QuickChat is to offer a seamless platform for real-time, room-based communication. Our objectives include:

User-Friendly Interface: Enable users to sign in with Google, create or join chat rooms, and exchange messages effortlessly.

Organized Room Management: Provide dedicated rooms with human-readable IDs for streamlined discovery and sharing.

Modern Tech Stack: Leverage React.js, Tailwind CSS, DaisyUI, and Firebase for a fast, reliable chat experience.

Key Features:-

Google Authentication: Secure, one-click sign-in via Firebase Authentication.

Room-Based Messaging: Create or join rooms using short, memorable IDs.

Live Synchronization: Instant message delivery powered by Cloud Firestore.

User Avatars: Display names and profile photos alongside messages.

Responsive UI: Clean, mobile-friendly interface built with React and Tailwind CSS.

Easy Sharing: One-click copy for room IDs to invite others.

PRE-REQUISITES:-

Here are the key prerequisites for developing QuickChat using React.js and Firebase:

Node.js and npm:

Node.js is a powerful JavaScript runtime environment that allows you to run JavaScript code on the local environment. It provides a scalable and efficient platform for building network applications.

Install Node.js and npm on your development machine, as they are required to run JavaScript on the server-side.

Download: <https://nodejs.org/en/download/>

Installation instructions: <https://nodejs.org/en/download/package-manager/>

React.js:

React.js is a popular JavaScript library for building user interfaces. It enables developers to create interactive and reusable UI components, making it easier to build dynamic and responsive web applications.

Install React.js, a JavaScript library for building user interfaces.

Create a new React app:

```
npm create vite@latest
```

Enter and then type project-name and select React as the framework, then enter.

Navigate to the project directory:

```
cd project-name
```

```
npm install
```

Running the React App:

With the React app created, you can now start the development server and see your React application in action.

Start the development server:

```
npm run dev
```

This command launches the development server, and you can access your React app at <http://localhost:5173> in your web browser.

HTML, CSS, and JavaScript:

Basic knowledge of HTML for creating the structure of your app, CSS for styling, and JavaScript for client-side interactivity is essential.

Firebase Account:

QuickChat uses Firebase for authentication and real-time messaging. Create a Firebase project and obtain configuration credentials.

Firebase Console: <https://console.firebase.google.com>

Enable Authentication (Google sign-in) and Firestore Database in your Firebase project.

Version Control:

Use Git for version control, enabling collaboration and tracking changes throughout the development process. Platforms like GitHub or Bitbucket can host your repository.

Git: Download and installation instructions can be found at: <https://git-scm.com/downloads>

Development Environment:

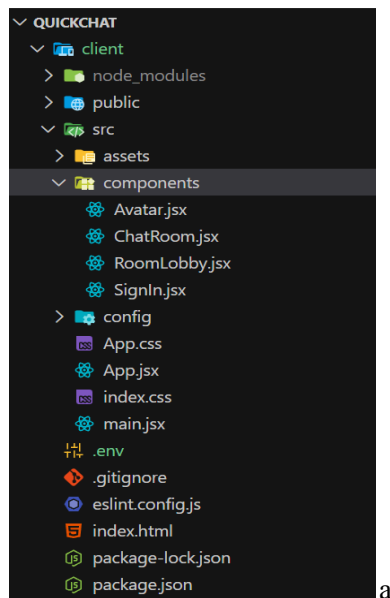
Choose a code editor or Integrated Development Environment (IDE) that suits your preferences, such as Visual Studio Code, Sublime Text, or WebStorm.

Visual Studio Code: Download from <https://code.visualstudio.com/download>

Sublime Text: Download from <https://www.sublimetext.com/download>

WebStorm: Download from <https://www.jetbrains.com/webstorm/download>

Project Structure:



The project structure may vary depending on the specific library, framework, programming language, or development approach used. It's essential to organize the files and directories in a logical and consistent manner to improve code maintainability and collaboration among developers.

src/App.jsx and src/App.css:

These files define the main App component, which serves as the root component for QuickChat. The component handles authentication state, manages the current room ID, and includes React Router configuration for loading different components (SignIn, RoomLobby, ChatRoom) based on the current route and user state.

PROJECT FLOW:-

Project Demo:

Before starting to work on this project, let's see the demo.

Demo link: <https://quick-chat-qc.vercel.app/>

Use the code in:

Milestone 1: Project Setup and Configuration

1. Install required tools and software:

Installation of required tools:

Open the project folder to install necessary tools.

In this project, we use:

- React.js
- React Router
- Firebase (Authentication & Firestore)
- Tailwind CSS
- DaisyUI

For further reference, use the following resources:

- <https://react.dev/learn/installation>
- <https://tailwindcss.com/docs/installation>
- <https://firebase.google.com/docs/web/setup>
- <https://daisyui.com/docs/install/>

Milestone 2: Web Development

1. Setup React Application:

- Create React application using Vite.
- Configure Routing with React Router.
- Install required libraries (Firebase, Tailwind CSS, DaisyUI).

App.jsx component:

```
import { useState, useEffect } from 'react';
import { onAuthStateChanged } from 'firebase/auth';
import { Routes, Route, useNavigate, Navigate } from 'react-router-dom';
import { auth } from '../config/firebase';
import SignIn from '../components/SignIn';
import RoomLobby from '../components/RoomLobby';
import ChatRoom from '../components/ChatRoom';

export default function App() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [roomId, setRoomId] = useState('');
  const navigate = useNavigate();

  // listening for auth state changes
  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (currentUser) => {
      setUser(currentUser);
      setLoading(false);
      if (!currentUser) {
        navigate('/signin', { replace: true });
      }
    });
    return () => unsubscribe();
  }, [navigate]);

  const handleCreateRoom = (newRoomId) => {
    setRoomId(newRoomId);
    navigate('/chat');
  }
}
```

```

const handleJoinRoom = (roomIdToJoin) => {
  setRoomId(roomIdToJoin);
  navigate('/chat');
};

const handleLeaveRoom = () => {
  setRoomId('');
  navigate('/join');
};

if (loading) {
  return (
    <div className="min-h-screen bg-base-200 flex items-center justify-center">
      <span className="loading loading-dots loading-lg"></span>
    </div>
  );
}

return (
  <Routes>
    <Route
      path="/signin"
      element={
        !user ? <SignIn /> : <Navigate to="/join" replace />
      }
    />
  </Routes>
);

```

```

    element={
      user ? (
        <RoomLobby
          user={user}
          onCreateRoom={handleCreateRoom}
          onJoinRoom={handleJoinRoom}
        />
      ) : (
        <Navigate to="/signin" replace />
      )
    }
  />
  <Route
    path="/chat"
    element={
      user && roomId ? (
        <ChatRoom
          user={user}
          roomId={roomId}
          onLeaveRoom={handleLeaveRoom}
        />
      ) : (
        <Navigate to={user ? "/join" : "/signin"} replace />
      )
    }
  />
  <Route path="*" element={<Navigate to={user ? "/join" : "/signin"} replace />} />
</Routes>

```

Code Description:-

- ./App.css: The CSS file for styling the App component.
- SignIn from ./components/SignIn: The SignIn component for Google authentication.
- RoomLobby from ./components/RoomLobby: The lobby component for creating/joining rooms.
- ChatRoom from ./components/ChatRoom: The main chat interface component.
- auth from ./config/firebase: Firebase authentication instance.
- The App function is defined, which manages authentication state and routing.
- Uses useState to track the current user, loading state, and active room ID.
- Uses useEffect with onAuthStateChanged to listen for authentication state changes.
- Use useNavigate from React Router to handle navigation between routes.
- Implements route guards to redirect users based on authentication and room state.
- Returns JSX with Routes and Route components for navigation:
 - /signin: Displays the SignIn component for unauthenticated users.
 - /join: Displays RoomLobby for authenticated users to create/join rooms.
 - /chat: Displays ChatRoom when user is authenticated and has joined a room.
 - *: Catch-all route that redirects based on user state.
- The App component is exported as the default export of the module.

2. Design UI components:

Create Components (SignIn, RoomLobby, ChatRoom, Avatar).

Implement layout and styling using Tailwind CSS and DaisyUI.

Add navigation with React Router.

3. Implement frontend logic:

Integration with Firebase Authentication API.

Integration with Firestore for real-time messaging.

Implement data binding for messages and user state.

Firestore Configuration Component:-

```
import { initializeApp } from 'firebase/app'
import { getAuth } from 'firebase/auth'
import { getFirestore } from 'firebase/firestore'

const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
  measurementId: import.meta.env.VITE_FIREBASE_MEASUREMENT_ID,
}

// Initialize Firebase
const app = initializeApp(firebaseConfig)
export const auth = getAuth(app)
export const db = getFirestore(app)
```

Code Description:-

- Imports initializeApp from Firebase to initialize the app.
- Imports getAuth from Firebase Authentication.
- Imports getFirestore from Firebase Firestore.
- Creates a firebaseConfig object with environment variables from .env file.
- Initializes Firebase app using initializeApp(firebaseConfig).
- Creates auth instance using getAuth(app) for authentication.
- Creates a db instance using getFirestore(app) for Firestore database.
- Exports auth and db for use throughout the application.

SignIn Component:-

```
import { signInWithPopup, GoogleAuthProvider } from 'firebase/auth';
import { auth } from '../config/firebase';

export default function SignIn() {
  const handleGoogleSignIn = async () => {
    const provider = new GoogleAuthProvider();
    try {
      await signInWithPopup(auth, provider);
    } catch (error) {
      console.error('Error signing in:', error);
    }
  };

  return (
    <div className="min-h-screen bg-base-200 flex items-center justify-center p-4">
      <div className="card w-96 bg-base-100 shadow-2xl">
        <div className="card-body text-center">
          <h1 className="card-title text-4xl font-bold justify-center mb-2">QuickChat</h1>
          <p className="text-base-content/70 mb-4">Sign in to start chatting with anyone!</p>
          <button
            onClick={handleGoogleSignIn}
            className="btn btn-outline gap-2">
            
            Sign in with Google
          </button>
        </div>
      </div>
    </div>
  );
}
```

Code Description:-

- Imports GoogleAuthProvider and signInWithPopup from Firebase Authentication.
- Imports auth from the Firebase configuration file.
- Defines the SignIn component which handles Google authentication.
- Creates a signInWithGoogle function that:
- Creates a new GoogleAuthProvider instance.
- Calls signInWithPopup(auth, provider) to trigger Google sign-in flow.
- Handles errors with try-catch block.
- Returns a styled card layout using DaisyUI classes.
- Renders a "Sign in with Google" button that calls signInWithGoogle on click.
- Exports the SignIn component as the default export.

RoomLobby Component:-

```
const handleJoinRoom = () => {
  if (roomIdInput.trim()) {
    onJoinRoom(roomIdInput);
  }
};

return (
  <div className="min-h-screen bg-base-200 flex items-center justify-center p-4">
    <div className="card w-96 bg-base-100 shadow-2xl">
      <div className="card-body">
        <div className="flex items-center justify-between mb-4">
          <div className="flex items-center gap-3">
            <Avatar photoURL={user.photoURL} displayName={user.displayName} size="md" />
            <span className="font-semibold">{user.displayName}</span>
          </div>
          <button onClick={handleSignOut} className="btn btn-ghost btn-sm border-1 border-gray-300">
            Sign out
          </button>
        </div>
        <h2 className="card-title text-2xl justify-center mb-4">Join or Create Room</h2>

        <div className="space-y-4">
          <button
            onClick={handleCreateRoom}
            className="btn btn-primary w-full"
          >
            Create New Room

```

```

          <div className="divider">OR</div>

          <div className="form-control gap-2 ">
            <input
              type="text"
              placeholder="Enter Room ID"
              value={roomIdInput}
              onChange={(e) => setRoomIdInput(e.target.value)}
              className="input input-bordered w-full mb-5"
            />
            <button
              onClick={handleJoinRoom}
              disabled={!roomIdInput.trim()}
              className="btn btn-secondary w-full"
            >
              Join Room
            </button>
          </div>
        </div>
      </div>
    </div>
  </div>
);
}
```

Code Description:-

- Imports useState from React to manage component state.
- Imports signOut from Firebase Authentication.
- Imports auth from Firebase configuration.
- Imports Avatar component for displaying user profile.
- Accepts props: user, onCreateRoom, onJoinRoom.
- Uses useState to manage roomIdInput for the join room input field.
- Defines handleCreateRoom function that:
- Generates a random room ID using adjective-noun-number pattern.
- Calls onCreateRoom prop with the new room ID.
- Defines handleJoinRoom function that validates and calls onJoinRoom with the input room ID.
- Defines handleSignOut function that calls signOut(auth).
- Returns a responsive layout with:
- User info section displaying avatar and name.
- Sign out button.
- Create room button.
- Join room input field and button.
- Exports the RoomLobby component as default.

ChatRoom Component:-

```
import { useState, useEffect, useRef } from 'react';
import { collection, addDoc, query, where, orderBy, onSnapshot, serverTimestamp } from 'firebase/firestore';
import { db } from '../config/firebase';
import Avatar from '../Avatar';

export default function ChatRoom({ user, roomId, onLeaveRoom }) {
  const [messages, setMessages] = useState([]);
  const [newMessage, setNewMessage] = useState('');
  const [toast, setToast] = useState(null);
  const messagesEndRef = useRef(null);

  // Listen for messages in the current room
  useEffect(() => {
    if (!roomId) return;

    const q = query(
      collection(db, 'messages'),
      where('roomId', '==', roomId),
      orderBy('timestamp', 'asc')
    );

    const unsubscribe = onSnapshot(q, (snapshot) => {
      const msgs = [];
      snapshot.forEach((doc) => {
        msgs.push({ id: doc.id, ...doc.data() });
      });
      setMessages(msgs);
    }, (error) => {
      console.error('Error listening to messages:', error);
    });

    console.error('Firestore index required. Check the error message for the link to create it.');
```

```
  }
  });

  return () => unsubscribe();
}, [roomId]);

// Auto scroll to bottom
useEffect(() => {
  messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
}, [messages]);

const sendMessage = async (e) => {
  e.preventDefault();
  if (!newMessage.trim()) return;

  try {
    await addDoc(collection(db, 'messages'), {
      text: newMessage,
      roomId: roomId,
      userId: user.uid,
      userName: user.displayName,
      userPhoto: user.photoURL,
      timestamp: serverTimestamp()
    });
  }
```



```

    setNewMessage('');
  } catch (error) {
    console.error('Error sending message:', error);
  }
};

const copyRoomId = () => {
  navigator.clipboard.writeText(roomId);
  setToast({ type: 'success', message: 'Room ID copied!' });
  setTimeout(() => setToast(null), 2000);
};

const handleLeaveRoom = () => {
  setToast({ type: 'info', message: 'Leaving the room...' });
  setTimeout(() => {
    setToast(null);
    onLeaveRoom();
  }, 1200);
};

return (
  <div className="h-screen bg-base-200 flex flex-col">
    <div className="navbar bg-base-100 shadow-md">
      <div className="flex-1 flex items-center gap-3">
        <Avatar photoURL={user.photoURL} displayName={user.displayName} size="md" />
        <div className="font-semibold">{user.displayName}</div>
      </div>
    </div>
  </div>

```

```

    <div className="badge badge-ghost badge-sm">Room: {roomId}</div>
    <button
      onClick={copyRoomId}
      className="btn btn-success btn-sm border border-black"
    >
      Copy ID
    </button>
    <button
      onClick={handleLeaveRoom}
      className="btn btn-error btn-sm border border-black"
    >
      Leave
    </button>
  </div>
</div>

{/* Toast notification */}
{toast && (
  <div className="toast toast-end toast-middle z-50">
    <div className={`alert ${toast.type === 'success' ? 'alert-success' : toast.type === 'info' ? 'alert-in`}
      <span>{toast.message}</span>
    </div>
  </div>
)}

{/* body(messages) */}
<div className="flex-1 overflow-y-auto p-4">
  {messages.length === 0 ? (

```

```

{messages.length === 0 ? (
  <div className="hero min-h-full">
    <div className="hero-content text-center">
      <div className="max-w-md">
        <h1 className="text-2xl font-bold">No messages yet!</h1>
        <p className="py-4">Share the room ID with your friend to start chatting.</p>
      </div>
    </div>
  </div>
) : (
  <div className="space-y-4">
    {messages.map((msg) => (
      <div key={msg.id} className={`chat ${msg.userId === user.uid ? 'chat-end' : 'chat-start'}`}>
        <div className="chat-image avatar">
          <div className="w-10 rounded-full">
            {msg.userPhoto ? (
              <img
                src={msg.userPhoto}
                alt={msg.userName}
                referrerPolicy="no-referrer"
                crossOrigin="anonymous"
                onError={(e) => {
                  const fallback = document.createElement('div');
                  fallback.className = 'w-10 h-10 rounded-full bg-primary text-primary-content flex items-center justify-center';
                  fallback.textContent = msg.userName?.[0]?.toUpperCase();
                  e.target.parentElement.replaceChild(fallback, e.target);
                }}
              />
            ) : (
              <div className="w-10 h-10 rounded-full bg-primary text-primary-content flex items-center justify-center">
                {msg.userName?.[0]?.toUpperCase()}
              </div>
            )}
          </div>
        </div>
      </div>
    ))}
  </div>
)
}

```

```

        {msg.userName?.[0]?.toUpperCase()}
      </div>
    ))
  </div>
</div>
<div className="chat-header opacity-50 mb-1">
  {msg.userName}
</div>
<div className={`chat-bubble ${msg.userId === user.uid ? 'chat-bubble-primary' : ''}`}>
  {msg.text}
</div>
</div>
)))
<div ref={messagesEndRef} />
</div>
)}
</div>

<form onSubmit={sendMessage} className="bg-base-100 p-4 border-t border-base-300">
  <div className="flex gap-2">
    <input
      type="text"
      value={newMessage}
      onChange={(e) => setNewMessage(e.target.value)}
      placeholder="Type a message..."
      className="input input-bordered flex-1"
    />
    <button
      type="submit"
      disabled={!newMessage.trim()}
      className="btn btn-primary"
    >
      Send
    </button>
  </div>
</form>
</div>
);
}

```

Code Description:-

- Imports useState, useEffect, useRef from React for state and lifecycle management.
- Imports Firestore functions: collection, addDoc, query, where, orderBy, onSnapshot, serverTimestamp.
- Imports db from Firebase configuration.
- Imports Avatar component for message avatars.
- Accepts props: user, roomId, onLeaveRoom.
- Uses useState to manage messages, newMessage, and toast state.
- Uses useRef for messagesEndRef to enable auto-scroll.
- Uses useEffect to:

- Set up Firestore listener for messages filtered by roomId and ordered by timestamp.
- Handle Firestore errors (e.g., missing index).
- Clean up listener on component unmount.
- Defines sendMessage function that:
- Validates message is not empty.
- Adds document to Firestore messages collection with message data.
- Clears input field after sending.
- Defines copyRoomId function to copy room ID to clipboard.
- Defines handleLeaveRoom function that calls onLeaveRoom prop.
- Auto-scrolls to bottom when new messages arrive.
- Returns a full-height layout with:
- Navbar showing user info, room ID, and action buttons.
- Messages list with sender avatars and formatted timestamps.
- Message input field and send button.
- Toast notifications for user actions.
- Exports the ChatRoom component as default.

Avatar Component:-

```
export default function Avatar({ photoURL, displayName, size = 'md' }) {
  const sizeClass = size === 'sm' ? 'w-8' : size === 'lg' ? 'w-12' : 'w-10';

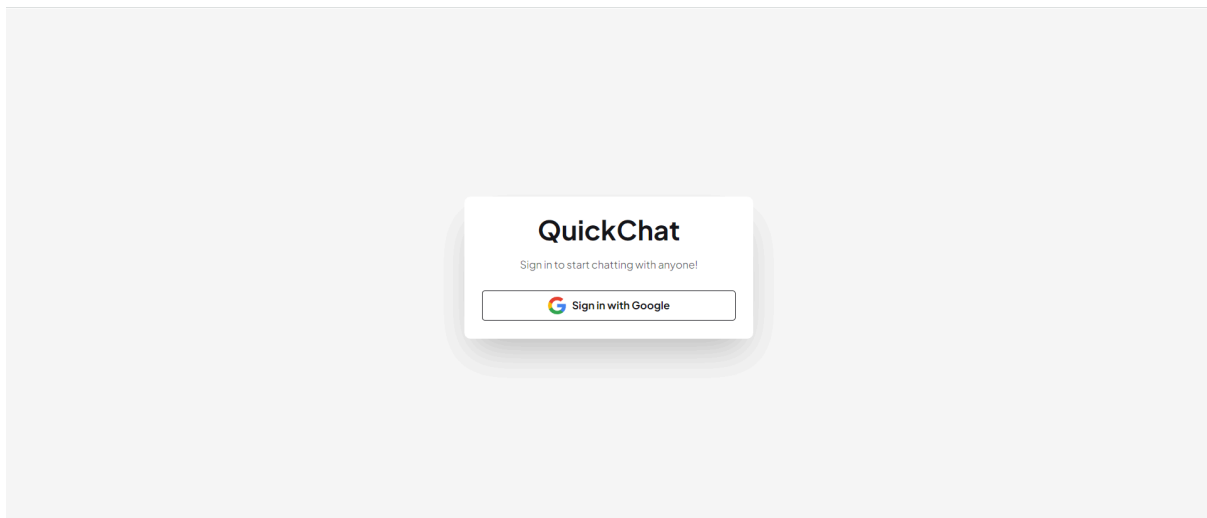
  return (
    <div className={`avatar ${!photoURL ? 'placeholder' : ''}`}>
      <div className={` ${sizeClass} rounded-full ${!photoURL ? 'bg-primary text-primary-content' : ''}`>
        {photoURL ? (
          <img
            src={photoURL}
            alt={displayName}
            referrerPolicy="no-referrer"
            crossOrigin="anonymous"
            onError={(e) => {
              e.target.style.display = 'none';
              e.target.parentElement.classList.add('bg-primary', 'text-primary-content');
              e.target.parentElement.innerHTML = `<span>${displayName?.[0]?.toUpperCase()}</span>`;
            }}
          />
        ) : (
          <span>{displayName?.[0]?.toUpperCase()}</span>
        )}
      </div>
    </div>
  );
}
```

Code Description:-

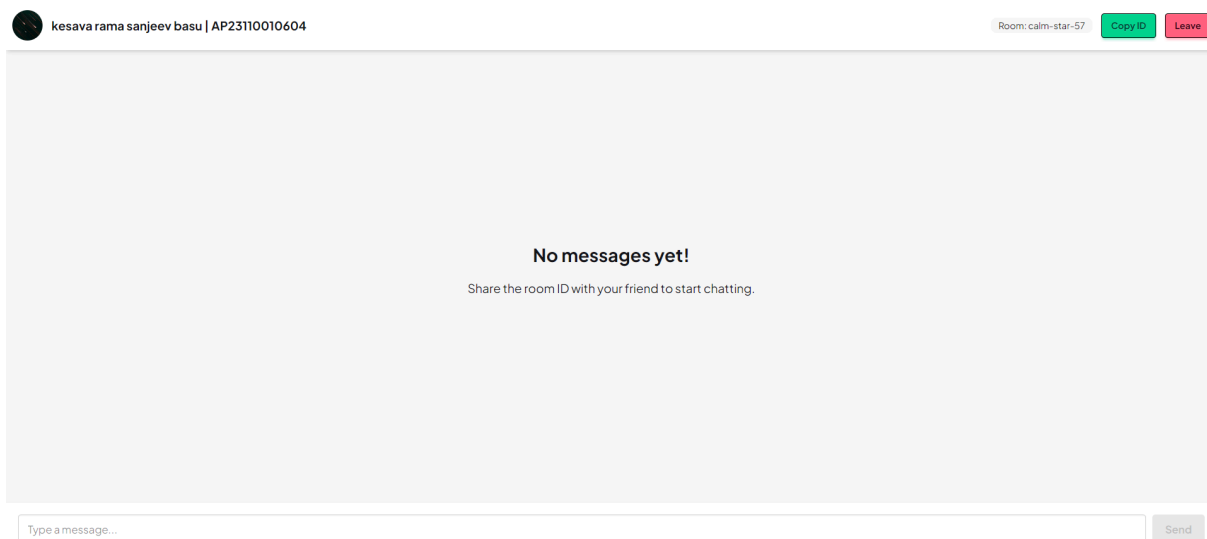
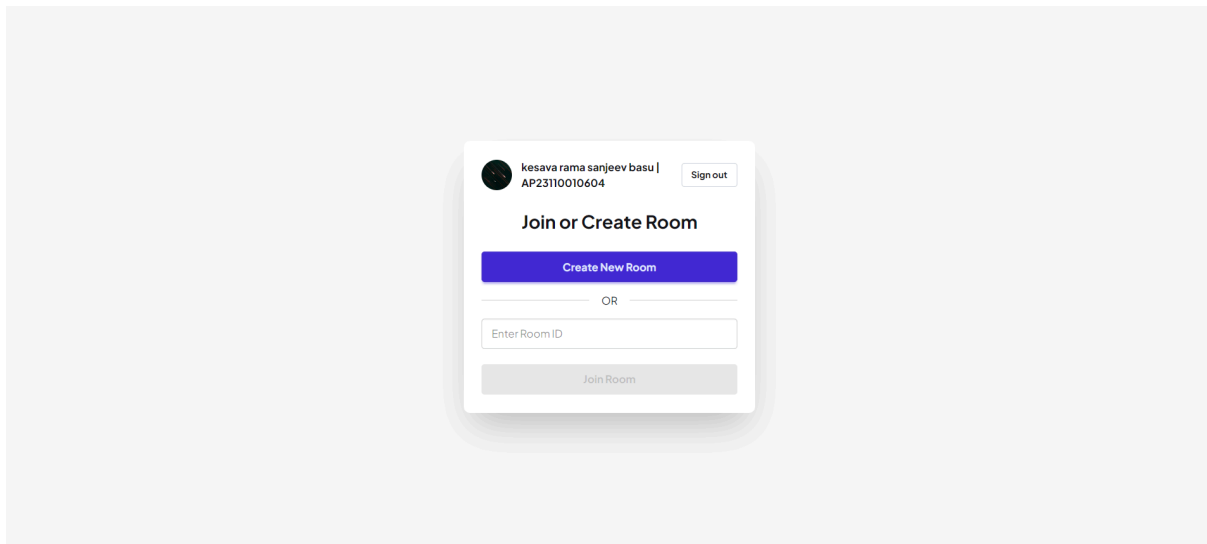
- Imports useState from React to manage image error state.
- Accepts props: photoURL, displayName, size.
- Uses useState to track imageError state.
- Defines handleImageError to set imageError to true on load failure.
- Returns a div with size-based classes (sm, md, lg).
- Conditionally renders:
- Image if photoURL exists and no error occurred.
- Fallback circle with first letter of displayName if no photo or error.
- Exports the Avatar component as default.

Project Execution:

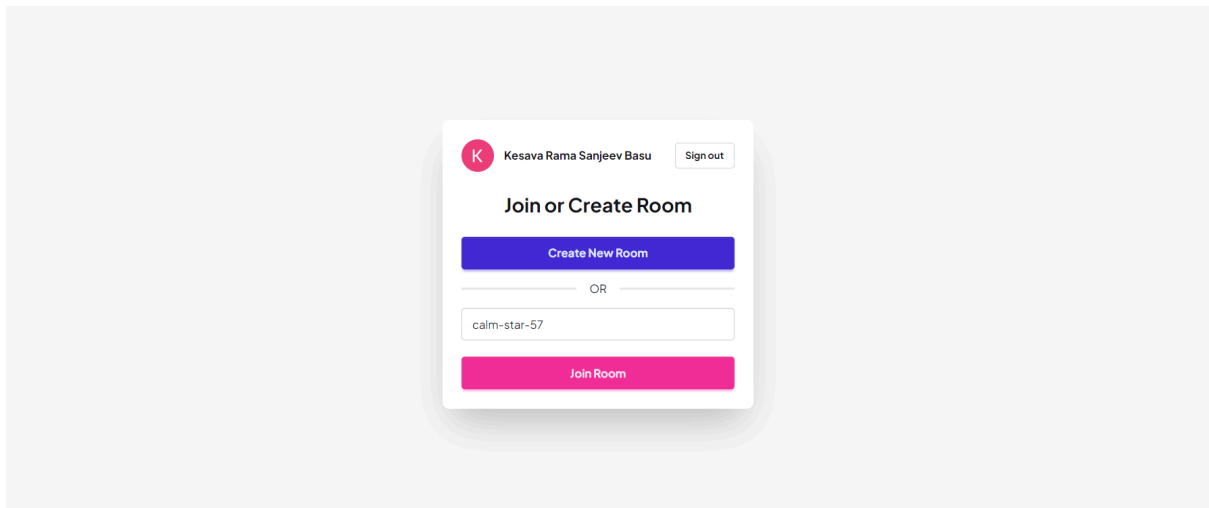
1. Sign In:
 - a. Click "Sign in with Google"
 - b. Authenticate with your Google account



2. Create a Room:
 - a. After sign-in, you'll be redirected to the Room Lobby
 - b. Click "Create New Room"
 - c. A room ID like sunny-fox-42 will be generated
 - d. You'll be redirected to the chat room

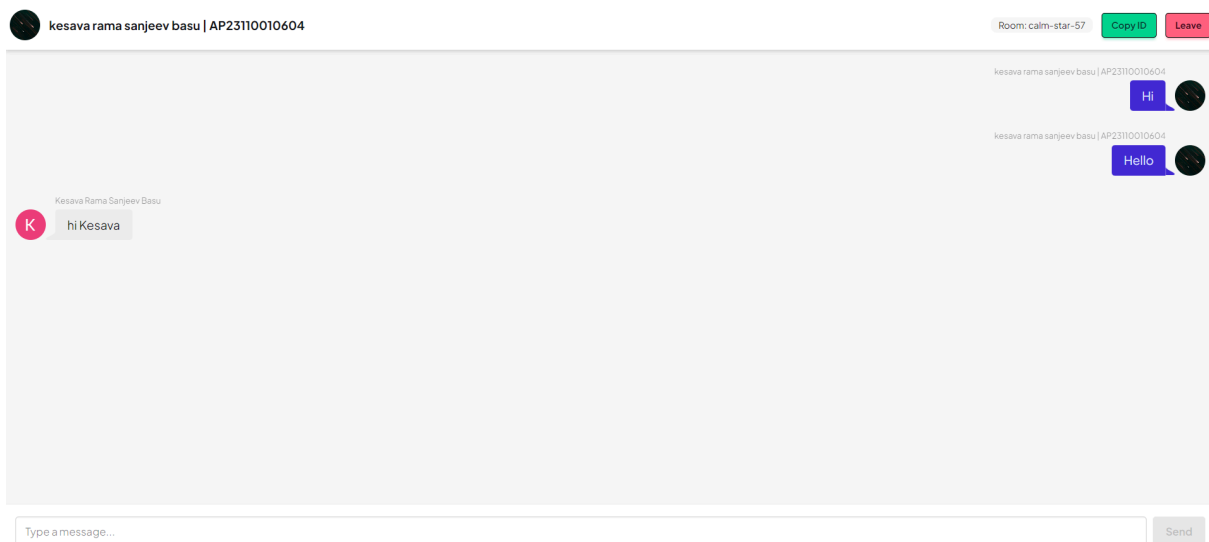


3. Join a Room:
 - a. Open the app in another browser/tab or share the room ID with someone
 - b. Enter the room ID in the "Join Room" input field
 - c. Click "Join Room"
 - d. Start chatting in real-time



4. Send Messages:

- Type a message in the input field at the bottom
- Press Enter or click "Send"
- Messages appear instantly for all participants



5. Leave Room:

- Click the "Leave Room" button in the navbar
- You'll be redirected back to the Room Lobby

K

Kesava Rama Sanjeev Basu

Room: calm-star-57

Copy ID

Leave

kesava rama sanjeev basu | AP2310010604

Hi

kesava rama sanjeev basu | AP2310010604

Hello

Kesava Rama Sanjeev Basu

hi Kesava

Leaving the room...

Type a message...

Send